IOTA JU Website Redesign
Orchestrator Guide

Project coordination and release responsibility

21 October 2026

1 Purpose and project status

This guide is for the person coordinating the four implementation handbooks. It defines decisions, handoffs, approvals and release gates that belong to the orchestrator. The four member documents define the code and evidence each member owns.

The new project folder and Git repository are named `iota-ju-next`. The repository contains planning documents at this stage; the application scaffold has not been created. Sourasish owns the first scaffold task. The existing website remains the source of truth until migration and launch are explicitly approved.

Architecture selected for planning: one Next.js App Router application using TypeScript and Node.js, with Payload CMS in the same app. The public website, Payload admin, authentication and content APIs deploy together to Vercel. PostgreSQL and object storage are separate managed services. No separate Express or FastAPI content server is planned. Exact package versions, database provider, storage provider and spending limit are chosen and recorded before provisioning.

2 The four member assignments

Member	Owns	Main areas
Sourasish	Platform and release integration	Next/Payload foundation, shared root configuration, public content readers, preview, cache, redirects, deployment and operations.
Ritendu	CMS, permissions and data	Payload collections and globals, access rules, generated types, migrations, relationships and editor workflow.
Pritish	Design and public pages	Approved design specification, public layouts and templates, responsive behavior, accessibility and page metadata. His handbook uses smaller step-by-step checklists and plain explanations.

Shreyam	Inventory, migration and media	Source manifest, media storage and delivery, import scripts, original preservation, verification and handover runbook.

Each member writes and runs checks for their own work. The assigned member submits the PR and evidence. Another member reviews it. The orchestrator decides scope and resolves cross-owner questions; that does not make the orchestrator the default implementer or sole tester.

3 Your responsibilities

1. **Keep the shared goal stable.** Confirm that the new site preserves approved wording, names, departments, roles, project order, reports, public links, images and existing club videos. Keep the redesign focused on clearer use and efficient delivery.
2. **Set gates and priorities.** Create or maintain the task board using the IDs in the four handbooks. Do not let downstream work depend on undocumented field names, URLs or media props.
3. **Make owner decisions.** Approve one visual direction before final styling. Confirm content corrections, identity merges, dates/statuses and retirement responses when source evidence is ambiguous.
4. **Control external services.** Confirm the club-owned Vercel account, database and object-storage providers, region, budget, access owners and billing before anyone creates paid resources. Keep staging and production data separate.
5. **Protect the existing site.** The new application is separate. Keep the old deployment and its data available for rollback. Excluding workshop Q&A from the new site does not authorize deleting the old service or its records.
6. **Integrate and accept evidence.** Review PRs, make sure migrations and shared files are serialized, deploy merged work to staging, and check the complete publishing flow. Ask the relevant owner to fix defects in their area.
7. **Authorize release.** Approve a production domain switch only after the release checklist, content reconciliation, restore rehearsal and rollback steps are complete. Keep the approval and evidence in the repository.

You do not own: the members' code, their unit tests, the CMS schema implementation, every content import, or routine PR fixes. Do not silently edit their workstreams to make a demo pass. Assign a focused follow-up to the owner.

4 Decisions you must track

Record each decision in a short issue or `docs/decisions/` note, with date, owner and affected handbook IDs.

- Repository remote and collaborator access. This folder is a local Git repository; no GitHub remote is configured by this work.
- Final visual direction. It is not selected yet. Pritish's F01 requires your approval before final page styling.

- Compatible Next.js, Payload, React, TypeScript and Node.js versions. Sourasish records the tested matrix and pins it in the lockfile. Payload's current official installation guidance requires Node.js 20.18.1 or newer and a compatible Next.js version; select a supported Node LTS and current compatible stable packages at kickoff.
- Vercel project/account and deployment environments; managed PostgreSQL and object storage/CDN provider, region, access controls and spend limit.
- Who may edit or publish CMS content, and who receives account recovery messages.
- Which uncertain source facts require a content-owner decision. Never invent a date, award, department, identity or current event status.
- The approved response for old workshop Q&A URLs. No new Q&A service is in scope.

5 Order of work and integration gates

Phase	**Work that can proceed**	**Gate you check**
A: Agree and inspect	Sourasish P01 starts after repository location is agreed; Ritendu B01, Pritish F01 and Shreyam M01 can prepare contracts/design/inventory in parallel.	Source revision and content contract are documented; theme is approved before final styling; no service spend yet.
B: Foundation	App and protected staging; CMS roles and first project model; shared public shell; media storage preparation and dry-run importer.	A sample project is private by default, and no production data or secrets are in a preview.
C: First complete slice	One real project flows from import to CMS draft, authorized preview, admin publish, public list/detail/home teaser and working video.	Demonstrate the entire flow on the merged staging deployment, including cache update and signed-out denial.
D: All content	Events, people/memberships, remaining projects, home/About/timeline, media components and public templates.	Every page uses real content; no fixture-only page; questionable source content is resolved or clearly recorded.
E: Readiness	Legacy redirects, mobile/accessibility/performance, source parity, backups, restore and editor guide.	Review the evidence checklist, not only a screenshot or successful HTTP response.
F: Release	Final content reconciliation, final import, production checks, domain switch and handover.	Your explicit approval is required before production routing changes.

6 How to run the team

1. Keep the current master plan and the four member handbooks in the repository. Put the task ID in each ticket and branch, for example `codex/m2-b03-projects`.
2. Ask each member to take one focused task at a time. The dependency lists in their handbooks are the ordering authority. Work can proceed in parallel only after the shared content/media interfaces are agreed.
3. Require each PR to state: task ID, files changed, acceptance criteria checked, exact commands run, staging link or screenshot when applicable, and known limits.
4. Have the file owner review shared files. Sourasish owns root package/configuration/CI and public-reader runtime. After P01, Ritendu owns Payload config, generated types and migration ordering. Shreyam owns Media storage pieces in coordination with Ritendu. Pritish owns site routes and presentation.
5. Merge schema migrations one at a time. Rebase a task on current main before final checks. Do not overwrite uncommitted work or regenerate shared types from a stale schema.
6. At the end of each phase, hold a short demo against merged staging and mark the gate in the task board. A branch-only demo does not prove integration.

7 Shared technical rules

- Public reads return published public fields only. Drafts and private media require an authenticated preview path. Never rely on hiding an admin button as authorization.
- Use Payload's server-side APIs from the Next.js app. Do not create a second backend or call the same app through its own HTTP endpoint.
- Upload large files directly to approved object storage with short-lived permissions. Vercel Functions have a 4.5 MB request/response body limit; the function must not proxy the site's full-size video files.
- Keep original videos and images intact. Create delivery copies and posters; do not store large original media in Git. A background-video loop is a prepared media output, not a request-time transcode.
- Use named tasks and tests from the member handbooks. The project targets are measured on an actual merged preview; proposed performance targets are not claims about current results.

8 Release acceptance checklist

[ ] Every inventoried public page/content item is migrated, intentionally redirected, retained or excluded with a recorded reason.

[ ] The old site remains recoverable; the new site has no workshop Q&A backend or copied participant/payment data.

[ ] Signed-out visitors cannot read drafts, admin data or private media; editors and admins pass the role matrix.

[ ] Public pages, legacy routes, photos, full demos, reports, embedded media and background footage work on a real device/browser.

[ ] Database and media backup restore is rehearsed in isolated staging. A code rollback is not described as a data restore.

[ ] Club owns the service accounts and recovery details, and members provide editor/developer handover notes.

[ ] You approve the production change and record the release commit, import report, verification evidence and rollback method.

9 Source notes

The task IDs and ownership are expanded in the four companion handbooks in this repository. Current setup references are the official Next.js installation guide and Payload installation guide. Vercel's current function limits document the 4.5 MB body limit. Checked 21 October 2026. Re-check version compatibility when P01 begins; do not copy an outdated version number into a new install command.